CHAPTER 2

RANDOM NUMBER, RANDOM VARIABLE, AND STOCHASTIC PROCESS GENERATION

2.1 INTRODUCTION

This chapter deals with the computer generation of random numbers, random variables, and stochastic processes. In a typical stochastic simulation, randomness is introduced into simulation models via independent uniformly distributed random variables. These random variables are then used as building blocks to simulate more general stochastic systems.

The rest of this chapter is organized as follows. We start, in Section 2.2, with the generation of uniform random variables. Section 2.3 discusses general methods for generating one-dimensional random variables. Section 2.4 presents specific algorithms for generating variables from commonly used continuous and discrete distributions. In Section 2.5 we discuss the generation of random vectors. Sections 2.6 and 2.7 treat the generation of Poisson processes, Markov chains and Markov jump processes. Finally, Section 2.8 deals with the generation of random permutations.

2.2 RANDOM NUMBER GENERATION

In the early days of simulation, randomness was generated by *manual* techniques, such as coin flipping, dice rolling, card shuffling, and roulette spinning. Later on, *physical devices*, such as noise diodes and Geiger counters, were attached to computers for the same purpose. The prevailing belief held that only mechanical or electronic devices could produce truly random sequences. Although mechanical devices are still widely used in gambling

Simulation and the Monte Carlo Method, Second Edition. By R.Y. Rubinstein and D. P. Kroese
Copyright © 2007 John Wiley & Sons, Inc.

and lotteries, these methods were abandoned by the computer-simulation community for several reasons: (a) Mechanical methods were too slow for general use, (b) the generated sequences cannot be reproduced and, (c) it has been found that the generated numbers exhibit both bias and dependence. Although certain modern physical generation methods are fast and would pass most statistical tests for randomness (for example, those based on the universal background radiation or the noise of a PC chip), their main drawback remains their lack of repeatability. Most of today's random number generators are not based on physical devices, but on simple algorithms that can be easily implemented on a computer. They are fast, require little storage space, and can readily reproduce a given sequence of random numbers. Importantly, a good random number generator captures all the important statistical properties of true random sequences, even though the sequence is generated by a deterministic algorithm. For this reason, these generators are sometimes called *pseudorandom*.

The most common methods for generating pseudorandom sequences use the so-called *linear congruential generators*, introduced in [6]. These generate a deterministic sequence of numbers by means of the recursive formula

$$X_{i+1} = aX_i + c \pmod{m}\,, \tag{2.1}$$

where the initial value, X_0, is called the *seed* and the a, c, and m (all positive integers) are called the *multiplier*, the *increment* and the *modulus*, respectively. Note that applying the modulo-m operator in (2.1) means that $aX_i + c$ is divided by m, and the remainder is taken as the value for X_{i+1}. Thus, each X_i can only assume a value from the set $\{0, 1, \ldots, m-1\}$, and the quantities

$$U_i = \frac{X_i}{m}\,, \tag{2.2}$$

called *pseudorandom numbers*, constitute approximations to a true sequence of uniform random variables. Note that the sequence $X_0, X_1, X_2, \ldots$ will repeat itself after at most m steps and will therefore be periodic, with period not exceeding m. For example, let $a = c = X_0 = 3$ and $m = 5$. Then the sequence obtained from the recursive formula $X_{i+1} = 3X_i + 3 \pmod 5$ is $3, 2, 4, 0, 3$, which has period 4. In the special case where $c = 0$, (2.1) simply reduces to

$$X_{i+1} = a\,X_i \pmod{m}\,. \tag{2.3}$$

Such a generator is called a *multiplicative congruential generator*. It is readily seen that an arbitrary choice of X_0, a, c, and m will not lead to a pseudorandom sequence with good statistical properties. In fact, number theory has been used to show that only a few combinations of these produce satisfactory results. In computer implementations, m is selected as a large prime number that can be accommodated by the computer word size. For example, in a binary 32-bit word computer, statistically acceptable generators can be obtained by choosing $m = 2^{31} - 1$ and $a = 7^5$, provided that the first bit is a sign bit. A 64-bit or 128-bit word computer will naturally yield better statistical results.

Formulas (2.1), (2.2), and (2.3) can be readily extended to pseudorandom vector generation. For example, the n-dimensional versions of (2.3) and (2.2) can be written as

$$\mathbf{X}_{i+1} = A\mathbf{X}_i \pmod{\mathbf{M}} \tag{2.4}$$

and

$$\mathbf{U}_i = \mathbf{M}^{-1}\mathbf{X}_i, \tag{2.5}$$

respectively, where A is a nonsingular $n \times n$ matrix, $\mathbf{M}$ and $\mathbf{X}$ are n-dimensional vectors, and $\mathbf{M}^{-1}\mathbf{X}_i$ is the n-dimensional vector with components $M_1^{-1}X_1, \ldots, M_n^{-1}X_n$.

Besides the linear congruential generators, other classes have been proposed to achieve longer periods and better statistical properties (see [5]).

Most computer languages already contain a built-in pseudorandom number generator. The user is typically requested only to input the initial seed, X_0, and upon invocation the random number generator produces a sequence of independent, uniform $(0, 1)$ random variables. We therefore assume in this book the availability of such a "black box" that is capable of producing a stream of pseudorandom numbers. In Matlab, for example, this is provided by the `rand` function.

■ EXAMPLE 2.1 Generating Uniform Random Variables in Matlab

This example illustrates the use of the `rand` function in Matlab to generate samples from the $\mathrm{U}(0, 1)$ distribution. For clarity we have omitted the "`ans =`" output in the Matlab session below.

```
>> rand                  % generate a uniform random number
   0.0196
>> rand                  % generate another uniform random number
   0.823
>> rand(1,4)             % generate a uniform random vector
   0.5252  0.2026  0.6721  0.8381
rand('state',1234)       % set the seed to 1234
>> rand                  % generate a uniform random number
   0.6104
rand('state',1234)       % reset the seed to 1234
>> rand
   0.6104                % the previous outcome is repeated
```

2.3 RANDOM VARIABLE GENERATION

In this section we discuss various general methods for generating one-dimensional random variables from a prescribed distribution. We consider the inverse-transform method, the alias method, the composition method, and the acceptance-rejection method.

2.3.1 Inverse-Transform Method

Let X be a random variable with cdf F. Since F is a nondecreasing function, the inverse function F^{-1} may be defined as

$$F^{-1}(y) = \inf\{x : F(x) \geqslant y\}\,, \quad 0 \leqslant y \leqslant 1\,. \tag{2.6}$$

(Readers not acquainted with the notion inf should read min.) It is easy to show that if $U \sim \mathrm{U}(0, 1)$, then

$$X = F^{-1}(U) \tag{2.7}$$

has cdf F. Namely, since F is invertible and $\mathbb{P}(U \leqslant u) = u$, we have

$$\mathbb{P}(X \leqslant x) = \mathbb{P}(F^{-1}(U) \leqslant x) = \mathbb{P}(U \leqslant F(x)) = F(x)\,. \tag{2.8}$$

Thus, to generate a random variable X with cdf F, draw $U \sim \mathsf{U}(0,1)$ and set $X = F^{-1}(U)$. Figure 2.1 illustrates the inverse-transform method given by the following algorithm.

Algorithm 2.3.1 (The Inverse-Transform Method)

1. *Generate U from* $\mathsf{U}(0,1)$.
2. *Return* $X = F^{-1}(U)$.

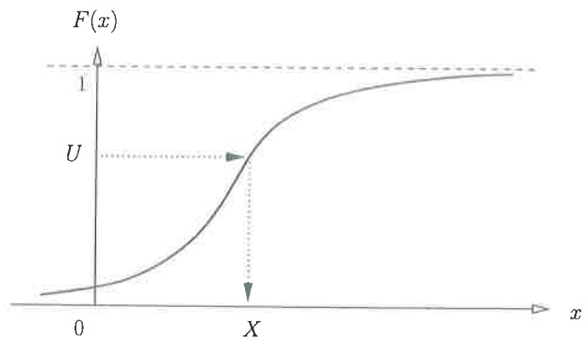

Figure 2.1 Inverse-transform method.

■ **EXAMPLE 2.2**

Generate a random variable from the pdf

$$f(x) = \begin{cases} 2x, & 0 \leqslant x \leqslant 1 \\ 0 & \text{otherwise.} \end{cases} \tag{2.9}$$

The cdf is

$$F(x) = \begin{cases} 0, & x < 0 \\ \int_0^x 2y\,dy = x^2, & 0 \leqslant x \leqslant 1 \\ 1, & x > 1. \end{cases}$$

Applying (2.7), we have

$$X = F^{-1}(U) = \sqrt{U}\,.$$

Therefore, to generate a random variable X from the pdf (2.9), first generate a random variable U from $\mathsf{U}(0,1)$ and then take its square root.

■ **EXAMPLE 2.3 Order Statistics**

Let $X_1, \ldots, X_n$ be iid random variables with cdf F. We wish to generate random variables $X_{(n)}$ and $X_{(1)}$ that are distributed according to the order statistics $\max(X_1, \ldots, X_n)$ and $\min(X_1, \ldots, X_n)$, respectively. From Example 1.7 we see

that the cdfs of $X_{(n)}$ and $X_{(1)}$ are $F_n(x) = [F(x)]^n$ and $F_1(x) = 1 - [1 - F(x)]^n$, respectively. Applying (2.7), we get

$$X_{(n)} = F^{-1}(U^{1/n})$$

and, since $1 - U$ is also from $\mathsf{U}(0,1)$,

$$X_{(1)} = F^{-1}(1 - U^{1/n})\,.$$

In the special case where $F(x) = x$, that is, $X_i \sim \mathsf{U}(0,1)$, we have

$$X_{(n)} = U^{1/n} \quad \text{and} \quad X_{(1)} = 1 - U^{1/n}\,.$$

■ **EXAMPLE 2.4 Drawing from a Discrete Distribution**

Let X be a discrete random variable with $\mathbb{P}(X = x_i) = p_i$, $i = 1, 2, \ldots$, with $\sum_i p_i = 1$ and $x_1 < x_2 < \ldots$. The cdf F of X is given by $F(x) = \sum_{i: x_i \leqslant x} p_i$, $i = 1, 2, \ldots$ and is illustrated in Figure 2.2.

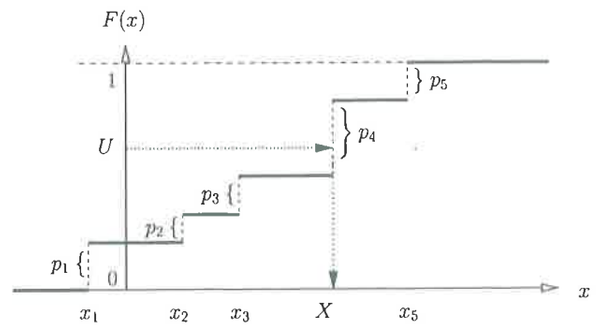

Figure 2.2 Inverse-transform method for a discrete random variable.

Hence, the algorithm for generating a random variable from F can be written as follows.

Algorithm 2.3.2 (Inverse-Transform Method for a Discrete Distribution)

1. *Generate* $U \sim \mathsf{U}(0,1)$.
2. *Find the smallest positive integer, k, such that $U \leqslant F(x_k)$ and return $X = x_k$.*

Much of the execution time in Algorithm 2.3.2 is spent in making the comparisons of Step 2. This time can be reduced by using efficient search techniques (see [2]).

In general, the inverse-transform method requires that the underlying cdf, F, exists in a form for which the corresponding inverse function F^{-1} can be found analytically or algorithmically. Applicable distributions are, for example, the exponential, uniform, Weibull, logistic, and Cauchy distributions. Unfortunately, for many other probability distributions, it is either impossible or difficult to find the inverse transform, that is, to solve

$$F(x) = \int_{-\infty}^{x} f(t)\,dt = u$$

■ **EXAMPLE 2.6**

Generate a random variable Z from the semicircular density

$$f(x) = \frac{2}{\pi R^2}\sqrt{R^2 - x^2}, \quad -R \leqslant x \leqslant R .$$

Take the proposal distribution to be uniform over $[-R, R]$, that is, take $g(x) = 1/(2R)$, $-R \leqslant x \leqslant R$ and choose C as small as possible such that $Cg(x) \geqslant f(x)$; hence, $C = 4/\pi$. Then Algorithm 2.3.6 leads to the following generation algorithm:

1. *Generate two independent random variables, U_1 and U_2, from* $\mathsf{U}(0,1)$.
2. *Use U_2 to generate X from $g(x)$ via the inverse-transform method, namely, $X = (2U_2 - 1)R$, and calculate*

$$\frac{f(X)}{C\,g(X)} = \sqrt{1 - (2U_2 - 1)^2} .$$

3. *If $U_1 \leqslant f(X)/(C\,g(X))$, which is equivalent to $(2U_2 - 1)^2 \leqslant 1 - U_1^2$, return $Z = X = (2U_2 - 1)R$; otherwise, return to Step 1.*

The expected number of trials for this algorithm is $C = 4/\pi$, and the efficiency is $1/C = \pi/4 \approx 0.785$.

2.4 GENERATING FROM COMMONLY USED DISTRIBUTIONS

The next two subsections present algorithms for generating variables from commonly used continuous and discrete distributions. Of the numerous algorithms available (for example, [2]), we have tried to select those that are reasonably efficient and relatively simple to implement.

2.4.1 Generating Continuous Random Variables

2.4.1.1 Exponential Distribution We start by applying the inverse-transform method to the exponential distribution. If $X \sim \mathsf{Exp}(\lambda)$, then its cdf F is given by

$$F(x) = 1 - \mathrm{e}^{-\lambda x}, \qquad x \geqslant 0 . \tag{2.16}$$

Hence, solving $u = F(x)$ in terms of x gives

$$F^{-1}(u) = -\frac{1}{\lambda}\ln(1-u) .$$

Noting that $U \sim \mathsf{U}(0,1)$ implies $1 - U \sim \mathsf{U}(0,1)$, we obtain the following algorithm.

Algorithm 2.4.1 (Generation of an Exponential Random Variable)

1. *Generate $U \sim \mathsf{U}(0,1)$.*
2. *Return $X = -\frac{1}{\lambda}\ln U$ as a random variable from* $\mathsf{Exp}(\lambda)$.

There are many alternative procedures for generating variables from the exponential distribution. The interested reader is referred to [2].

2.4.1.2 Normal (Gaussian) Distribution If $X \sim \mathsf{N}(\mu, \sigma^2)$, its pdf is given by

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}, \qquad -\infty < x < \infty , \tag{2.17}$$

where μ is the mean (or expectation) and σ^2 the variance of the distribution.

Since inversion of the normal cdf is numerically inefficient, the inverse-transform method is not very suitable for generating normal random variables, and other procedures must be devised. We consider only generation from $\mathsf{N}(0,1)$ (standard normal variables), since any random $Z \sim \mathsf{N}(\mu, \sigma^2)$ can be represented as $Z = \mu + \sigma X$, where X is from $\mathsf{N}(0,1)$. One of the earliest methods for generating variables from $\mathsf{N}(0,1)$ is the following, developed by Box and Müller.

Let X and Y be two independent standard normal random variables, so (X, Y) is a random point in the plane. Let (R, Θ) be the corresponding polar coordinates. The joint pdf $f_{R,\Theta}$ of R and Θ is given by

$$f_{R,\Theta}(r,\theta) = \frac{1}{2\pi}\mathrm{e}^{-r^2/2}\,r \quad \text{for } r \geqslant 0 \text{ and } \theta \in [0, 2\pi) .$$

This can be seen as follows. Specifying x and y in terms of r and θ gives

$$x = r\cos\theta \quad \text{and} \quad y = r\sin\theta . \tag{2.18}$$

The Jacobian of this coordinate transformation is

$$\det\begin{pmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{pmatrix} = \begin{vmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{vmatrix} = r .$$

The result now follows from the transformation rule (1.20), noting that the joint pdf of X and Y is $f_{X,Y}(x,y) = \frac{1}{2\pi}\mathrm{e}^{-(x^2+y^2)/2}$. It is not difficult to verify that R and Θ are independent, that $\Theta \sim \mathsf{U}[0, 2\pi)$, and that $\mathbb{P}(R > r) = \mathrm{e}^{-r^2/2}$. This means that R has the same distribution as $\sqrt{V}$, with $V \sim \mathsf{Exp}(1/2)$. Namely, $\mathbb{P}(\sqrt{V} > v) = \mathbb{P}(V > v^2) = \mathrm{e}^{-v^2/2}$, $v \geqslant 0$. Thus, both Θ and R are easy to generate and are transformed via (2.18) into independent standard normal random variables. This leads to the following algorithm.

Algorithm 2.4.2 (Generation of a Normal Random Variable: Box–Müller Approach)

1. *Generate two independent random variables, U_1 and U_2, from* $\mathsf{U}(0,1)$.
2. *Return two independent standard normal variables, X and Y, via*

$$\begin{aligned} X &= (-2\ln U_1)^{1/2}\cos(2\pi U_2) , \\ Y &= (-2\ln U_1)^{1/2}\sin(2\pi U_2) . \end{aligned} \tag{2.19}$$

An alternative generation method for $\mathsf{N}(0,1)$ is based on the acceptance-rejection method. First, note that in order to generate a random variable Y from $\mathsf{N}(0,1)$, one can first generate a positive random variable X from the pdf

$$f(x) = \sqrt{\frac{2}{\pi}}\,\mathrm{e}^{-x^2/2}, \qquad x \geqslant 0 , \tag{2.20}$$

and then assign to X a random sign. The validity of this procedure follows from the symmetry of the standard normal distribution about zero.